

1. 编译器的工作可以分成若干个阶段，请按顺序排列编译器工作的 7 个阶段。

- ①词法分析
- ②语法分析
- ③语义分析
- ④中间代码生成
- ⑤独立于机器的代码优化
- ⑥代码生成
- ⑦依赖于机器的代码优化

2. 编译器会检查每个算符的运算对象，看他们的类型是否适当，这叫做**类型检查**。类型检查一般在编译器阶段划分中的哪个阶段进行？ **语义分析**

3. 编译器阶段划分的哪个阶段会将**记号流**构造**成抽象语法树**？ **语法分析**

4. 编译器阶段划分的哪个阶段会**去掉源程序中的注释**？ **词法分析**

5. 编译器的**前端**主要包括了编译器阶段划分的哪几个阶段？

词法分析、语法分析、语义分析、中间代码生成

6. C 语言和汇编语言相比较而言谁是低级语言？为什么？

汇编语言，因为汇编语言更接近机器语言

7. X86 机器上原有一个 C 编译器(C→X86)，如果想得到一个 MIPS 机器上的 C 编译器(C→MIPS)，则需要进行如下工作：

(1) 先写一个(C→MIPS)的 C 源程序（源程序 A）

(2) 在 X86 机器上用(C→X86)编译器编译源程序 A 得到 X86 机器上的(C→MIPS)编译器。

(3) 在 X86 机器上用刚生成的(C→MIPS)编译器再编译源程序 A 一次，即得到 MIPS 机器上的(C→MIPS)编译器。

请问这个过程中有没有**交叉编译**？如果有，交叉编译发生在哪一步？

有，发生在 (3) 中

在一种计算机环境中运行的编译程序，能编译出在另外一种环境下运行的代码，我们就称这种编译器支持交叉编译。这个编译过程就叫交叉编译

1. 如下程序，在词法分析阶段，是否会对 `reture` 报错？下面哪项说法正确？

```
if (sum > 0) reture sum;
```

词法分析阶段不会报错，reture 会被识别为标识符返回。

2. 下列哪个正规式表示 $\Sigma = \{a, b\}$ 上 a 和 b 构成的所有串？

$a|b)^*$

3. 二进制偶数用正规式表示为：

$(0|1)^*0$

4. 在编译器中，词法分析和语法分析之间的典型关系是什么？

语法分析为主导，词法分析器作为语法分析器的子程序，被语法分析器调用。

5. 如果对下列源程序进行词法分析，请问词法分析器应该返回多少个记号？

①printf②(③ “Total=%d\n”④, ⑤score⑥)⑦;

7 个

1. Lex 根据 Lex 源程序生成的词法分析器源程序中，必须包含的两部分内容是什么？（AC）

A. 词法分析器驱动程序 yylex()

B. 辅助函数

C. 根据 Lex 源程序中正规式构造的 DFA 状态转换表。

D. 语法分析器驱动程序 yyparse()

2. Lex 生成的词法分析器使用什么原则匹配被选中的词法单元？（BD）

A. 如果词法单元可与若干正规式匹配，则优先选择排在后面的正规式

B. 最长匹配原则

C. 最短匹配原则

D. 如果词法单元可与若干正规式匹配，则优先选择排在前面的正规式

3. Lex 源程序中有两条正规式：A 可以匹配所有标识符（以字母开头的字母数字组合），B 可以匹配 While 关键字。如果想让输入文件中的 While 被识别为 While 关键字，则在 Lex 源程序中（D）

A. A, B 的位置与识别 While 无关。

B. 无论怎么放 While 都被随机识别。

C. 要把 A 放在 B 前面。

D. 要把 B 放在 A 前面。

4. Lex 源程序的正规式中 \. 表示什么？（D）

A. 除\n 以外的所有字符。

B. 所有字符。

C. 字符\和字符.的连接。

D. 字符.

1. 正规式可以用于描述括号配对。 错

2. 如果两个文法产生同样的语言，则称这两个文法等价。 对

3. 如果一个文法 G 是二义的，则它所表示的语言 L(G)也是二义的。 错

4. 正规式能描述的所有结构都可以用上下文无关文法来描述。 对

5. 组成上下文无关文法的四元组包括

- A. 终结符集合
- B. 开始符号
- C. 非终结符集合
- D. 产生式集合

1. 按照惯常定义的“与”“或”“非”的优先级和结合性改写下列文法，

$S \rightarrow S \text{ and } S \mid S \text{ or } S \mid \text{not } S \mid p \mid q \mid (S)$

得到的等价的非二义文法是： (B)

- A. $E \rightarrow E \text{ or } T \mid T$
 $T \rightarrow T \text{ and } F \mid F$
 $F \rightarrow \text{not } G$
 $G \rightarrow (E) \mid p \mid q$
- B. $E \rightarrow E \text{ or } T \mid T$
 $T \rightarrow T \text{ and } F \mid F$
 $F \rightarrow \text{not } F \mid (E) \mid p \mid q$
- C. $E \rightarrow E \text{ or } T \mid T$
 $T \rightarrow T \text{ and } F \mid F$
 $F \rightarrow \text{not } E \mid (E) \mid p \mid q$
- D. $E \rightarrow T \text{ or } E \mid T$
 $T \rightarrow F \text{ and } T \mid F$
 $F \rightarrow \text{not } F \mid (E) \mid p \mid q$

2. 对于左递归文法： $A \rightarrow Aa \mid b$ ，消除左递归后得到的文法是：(C)

- A. $A \rightarrow b A'$
 $A' \rightarrow a A' \mid a$
- B. $A \rightarrow A' b$
 $A' \rightarrow a A' \mid \varepsilon$
- C. $A \rightarrow b A'$
 $A' \rightarrow a A' \mid \varepsilon$
- D. $A \rightarrow b A'$
 $A' \rightarrow A' a \mid \varepsilon$

3. 上下文无关文法在 Chomsky 分类中属于 (2) 型文法

4. Chomsky 文法分类中 3 型文法又称为： (D)

- A. 短语文法
- B. 上下文有关文法
- C. 上下文无关文法
- D. 正规文法

5. 假设有如下完整的 if 语句:

if (a>0) then if (a>1) then b=1; else b=0;

按照正常语义, else 应该和语句中从左数第(二)个 then 配对?

1. 下列文法是 LL(1)文法。

$$S \rightarrow AB$$

$$A \rightarrow ab \mid \varepsilon$$

$$B \rightarrow ac$$

错

解: $FIRST(ab)=\{a\}$, $FOLLOW(A)=\{a\}$, 故不是 LL(1)文法

2. 若有文法:

$$S \rightarrow AB$$

$$A \rightarrow ab \mid \varepsilon$$

$$B \rightarrow c$$

则 $First(S)=?$

$FIRST(S) = \{a, c\}$

3. 若有文法:

$$S \rightarrow AB$$

$$A \rightarrow ab \mid \varepsilon$$

$$B \rightarrow c$$

则 $Follow(A)=?$

$FOLLOW(A) = \{c\}$

4. 给文法:

$$S \rightarrow (L) \mid a$$

$$L \rightarrow SL'$$

$$L' \rightarrow , SL' \mid \varepsilon$$

设计递归下降的预测分析器时, 为 L' 写的函数如下:

```

void Lprime ()
{
    if(lookahead=='('){
        match('(');
        S();
        Lprime();
    } else {
        (      )
    } else error();
}

```

请问空白处应填写下列哪条语句?

(C)

A. return;
B. if (lookahead=='(') return;
C. if (lookahead=='') return;
D. if (lookahead=='') { match('('); return; }

1. 非递归预测分析算法中，栈中存放的是：(D)

- A. 终结符号串
- B. 产生式
- C. 非终结符号串
- D. 文法符号串

2. 非递归预测分析算法中，如果栈顶是一个非终结符，则分析器的动作为：(C)

- A. 宣布分析成功
- B. 从栈顶弹出该非终结符
- C. 根据当前输入 a 和栈顶非终结符 X 访问分析表 M(X, a)
- D. 宣布分析失败

3. 在非递归预测分析的分析表中，空白表项表示 (A)

- A. 出错
- B. 分析成功
- C. 应该将输入符号放入栈中
- D. 应该将栈顶符号弹出

4. 在非递归预测分析时，假设采用紧急方式的错误恢复，且同步集合选用 Follow(A)中的元素。那么当出错时，若栈顶是 A，输入经抛弃若干符号后，指向同步记号 c ($c \in \text{Follow}(A)$)，此时错误恢复动作应为 (D)

- A. 弹出栈顶 A 并抛弃输入符号 c，恢复分析栈中 A 下面的那个符号。
- B. 恢复分析栈顶的 A
- C. 抛弃输入符号 c，恢复分析栈顶的 A
- D. 弹出栈顶 A，恢复分析栈中 A 下面的那个符号。

5. 在非递归预测分析时，如果采用紧急方式的错误恢复，当栈顶元素是终结符并且和当前输入符号不匹配时，一般怎样进行错误恢复？ (B)

- A. 弹出栈顶终结符，抛弃当前输入，给出“插入了该终结符”的错误恢复信息，并恢复分析
- B. 弹出栈顶的终结符，给出“插入了该终结符”的错误恢复信息，并恢复分析
- C. 抛弃当前输入，直到输入可以和栈顶元素匹配为止。
- D. 将栈顶元素修改为与输入符号相同的终结符。

1. 最左推导的逆过程是 (最右规约)
(最左规约) 也称规范归约。

1. 构造规范的 LR 分析表时, 在根据项目集族和识别活前缀的 DFA 填充动作-转移表的时候, 如果碰到归约项目, 应该填写 action(i, a)为按相应产生式进行归约。请问怎样选择对哪些 a 填写归约呢? (C)

A. a 是 Follow(产生式左部非终结符) 中的元素。

B. a 是所有输入符号。

C. a 是搜索符。

D. a 是 First (产生式右部) 中的元素。

2. 在构造规范的 LR 分析表时, 应该构造 (B) 集合作为识别活前缀的 DFA 的状态。

A. LR(0)项目

B. LR(1)项目

C. 搜索符

D. 同心项目

3.

为如下文法构造规范的 LR 分析表, 在构造第一个项目集时, 正确的 I_0 是:

$$\begin{aligned} E' &\rightarrow E \\ E &\rightarrow E + T \mid T \\ T &\rightarrow T * F \mid F \\ F &\rightarrow (E) \mid id \end{aligned}$$

答案:

(A)

A. $E' \rightarrow \cdot E, \$$

$E \rightarrow \cdot E + T, \$/+$

$E \rightarrow \cdot T, \$/+$

$T \rightarrow \cdot T * F, \$/+/*$

$T \rightarrow \cdot F, \$/+/*$

$F \rightarrow \cdot (E), \$/+/*$

$F \rightarrow \cdot id, \$/+/*$

B. $E' \rightarrow \cdot E, \$$

$E \rightarrow \cdot E + T, \$$

$E \rightarrow \cdot T, \$$

$T \rightarrow \cdot T * F, \$$

$T \rightarrow \cdot F, \$$

$F \rightarrow \cdot (E), \$$

$F \rightarrow \cdot id, \$$

C. $E' \rightarrow \cdot E, \$$

$E \rightarrow \cdot E + T, \$/+$

$E \rightarrow \cdot T, \$/+$

$T \rightarrow \cdot T * F, \$/+$

$T \rightarrow \cdot F, \$/+$

$F \rightarrow \cdot (E), \$/+$

$F \rightarrow \cdot id, \$/+$

D. $E' \rightarrow \cdot E, \$$
 $E \rightarrow \cdot E + T, \$ / + / *$
 $E \rightarrow \cdot T, \$ / + / *$
 $T \rightarrow \cdot T * F, \$ / + / *$
 $T \rightarrow \cdot F, \$ / + / *$
 $F \rightarrow \cdot (E), \$ / + / *$
 $F \rightarrow \cdot id, \$ / + / *$

3. 所有能用规范 LR 分析进行分析的文法也都能用 SLR 分析进行分析。 错

5. 无论是规范的 LR 分析还是 SLR 分析，分析算法都是一样的，分析表的形式也一样，不同点在于 LR 分析表的构造方法不同。 对

1. 同心的 LR(1)项目集就是指只有搜索符不同，其他部分都相同的 LR(1)项目集。 对

2. 在 LALR 分析时，合并同心项目集可能会导致移进-归约冲突。 错

3. 当输入串有错误时，LALR 分析可能会比规范的 LR 分析移进更多的输入符号。 错

4. 有些文法能进行 LALR 分析，但是不能进行规范 LR 分析。 错

5. 所有非 LR 的上下文无关文法都是二义的。 错

6. 对于二义文法 $E \rightarrow E + E \mid E * E \mid (E) \mid id$
 进行 SLR 分析时，某 LR(0)项目集如下：

$E \rightarrow E + E \cdot$
 $E \rightarrow E \cdot + E$
 $E \rightarrow E \cdot * E$

已知 Follow (E) = {+, *,), \$}

请问该项目集中是否存在冲突？如果存在冲突，是在什么情况下存在冲突？按照正常语义，应该怎样处理？下面说法正确的是： (C)

- A. 遇到*时，存在归约-归约冲突，应保留按 $E \rightarrow E + E$ 进行归约。
- B. 遇到*时，存在移进归约冲突，应该优先归约。
- C. 遇到+时，存在移进归约冲突，应该优先归约。
- D. 遇到+时，存在移进归约冲突，应该优先于移进。
- E. 无冲突。

1. 进行 LR 分析时，一旦访问到转移表的空白项，则表示分析出错。 错

解：LR 分析访问转移表时不可能访问到空白项，访问动作表时遇到空白才表示出错。

2. 进行 LR 分析时，如果在试图将某输入子串归约为 A 的过程中出错，那么采用紧急方式的错误恢复，实际上相当于假设被退栈的部分和被抛弃的输入已经成功分析出了 A，从而可以继续后面的分析。 对

3. Yacc 解决分析动作冲突的一般原则是：除另有说明外，(D)

A. 对于移进-归约冲突，优先于归约，对于归约-归约冲突，优先于后面的产生式。

B. 对于移进-归约冲突，优先于归约，对于归约-归约冲突，优先于先出现的产生式。

C. 对于移进-归约冲突，优先于移进，对于归约-归约冲突，优先于后面的产生式。

D. 对于移进-归约冲突，优先于移进，对于归约-归约冲突，优先于先出现的产生式。

3. 在实现 LR 分析的短语级错误恢复时，实际上是为动作表中每个出错条目编写一个错误恢复子程序，分析一旦进入该出错条目，就调用该错误恢复子程序。 对

5. Yacc 解决分析动作冲突时，如果定义了终结符的优先级和结合性，则如果产生移进归约冲突，下面说法正确的是： (BCD)

A. 如果归约的产生式的优先级高于移进的终结符的优先级，则进行移进。

B. 如果归约的产生式的优先级高于移进的终结符的优先级，则进行归约。

C. Yacc 中产生式的优先级与产生式中处在最右端的终结符的优先级相同。

D. 如果归约的产生式的优先级和移进的终结符的优先级相同，且产生式左结合，则进行归约。

E. Yacc 中产生式的优先级与产生式中处在最右端的文法符号的优先级相同。

F. 如果归约的产生式的优先级和移进的终结符的优先级相同，且产生右结合，则进行归约。

1. 一个语法制导定义不包含下列哪项： (C)

A. 基础文法（上下文无关文法）

B. 产生式对应的语义规则

C. 属性依赖图

D. 文法符号的属性

2. 语法制导定义中的一组语义规则如下：

$A \rightarrow XYZ \quad A.a = X.a + Y.a; Z.a = A.a$

下列说法正确的是： (D)

A. Z.a 是 Z 的综合属性

B. A.a 是 A 的继承属性

C. Z.a 是 A 的继承属性

D. A.a 是 A 的综合属性

4. 分析树中每个结点的所有属性（包括虚拟属性）都应该在属性依赖图中有一个结点，如果属性 a 依赖于属性 b，则依赖图中应该有一条从结点 a 指向结点 b 的有向边。 对

5. S 属性定义中的综合属性可以由自下而上的分析器在分析输入的同时完成计算。
对

5. S 属性定义中的继承属性不可以自下而上计算。 错

1. S 属性定义也是一种 L 属性定义。 对

2. L 属性定义就是只含有继承属性的语法制导定义。 错

3. 翻译方案和语法制导定义的区别就是：语法制导定义中的语义规则在翻译方案中叫做语义动作，它被放在大括号里，可以插在产生式右部的任何地方，以此表示动作执行的时机。
对

4. L 属性定义的翻译方案有可能在进行预测分析的同时进行属性的计算。 对

5. 能不能给一个 S 属性定义写一个翻译方案？如果能，怎样写？下面说法正确的是：(D)

A. 能，但首先要将 S 属性定义改写成 L 属性定义

B. S 属性定义无法写出翻译方案

C. 能。只需要将 S 属性定义中的语义规则用大括号括起，然后放在产生式右部的开头。

D. 能。只需要将 S 属性定义中的语义规则用大括号括起，然后放在产生式右部的末尾。

6. 关于 L 属性定义的自下而上计算，下面说错误的是：(C)

A. 如果翻译方案中嵌在产生式中间的语义动作中没有继承属性，只产生副作用，则可以通过添加标记非终结符的方式使得语义动作只出现在产生式右部的末端。

B. 如果继承属性由简单复写规则产生，但在栈上位置不可预测，则可通过添加标记非终结符的方式将文法变换成继承属性在栈上位置可以预测的情况。

C. 如果继承属性是综合属性的函数，则没有办法用自下而上的方式进行计算了。

D. L 属性定义如果想进行自下而上的计算，关键是通过简单的变换，使得语义动作出现在产生式右部的末端，从而可参考 S 属性定义的自下而上分析进行分析。

E. 如果继承属性由简单复写规则产生，且在栈上的位置可预测，则可以写出语义动作中只体现综合属性计算的翻译方案，引用继承属性时只需要到栈上指定位置去找。

题解：如果继承属性是综合属性的函数，则可以通过添加标记非终结符的方式使对继承属性的计算变为对标记非终结符综合属性的计算，从而转变成继承属性由简单复写规则产生的情况。

1. 变量的类型的本质是变量在程序执行期间的取值范围。 对
2. 由下面这条定型规则（该规则出自课本简单类型检查器规范）可以看出：如果想要得出 id 的类型是 T 的定型结论，必须要求定型环境中包含 id，且 id 的类型为 T。

$$\frac{\Gamma_1, id : T, \Gamma_2 \vdash \Diamond}{\Gamma_1, id : T, \Gamma_2 \vdash id : T}$$

对

3. 下列关于类型可靠的说法不正确的是： (C)
- A. 类型可靠可以保证通过了静态类型检查的程序在动态执行时不出现不会被捕获的错误。
- B. 如果某语言所有良类型的程序都是良行为的，则称该语言为类型可靠的。
- C. C 语言是类型可靠的语言。
- D. 类型可靠的语言一定是安全的语言。

4. 类型系统是一种逻辑系统，它通常应该包含三类要素，这三类要素包括 ()
- A. 断言
- B. 类型检查器
- C. 推理规则
- D. 类型表达式

5. 类型系统中的推理规则有三种形式，下面这条规则属于哪种形式的推理规则？ ()

$$\frac{\Gamma \vdash \Diamond}{\Gamma \vdash \text{truth} : \text{boolean}}$$

- A. 语法规则
- B. 环境规则
- C. 公理规则
- D. 定型规则

• 环境规则

$$(\text{Env } \emptyset) \quad \frac{}{\emptyset \vdash \Diamond}$$

• 语法规则

$$(\text{Type Bool}) \quad \frac{\Gamma \vdash \Diamond}{\Gamma \vdash \text{boolean}}$$

• 定型规则

$$(\text{Val } +) \quad \frac{\Gamma \vdash M : \text{int}, \Gamma \vdash N : \text{int}}{\Gamma \vdash M + N : \text{int}}$$

语法规则：

$$\text{(Type Product)} \quad \frac{\Gamma \vdash T_1, \Gamma \vdash T_2}{\Gamma \vdash T_1 \times T_2}$$

定型规则

$$\text{(Exp Pair)} \quad \frac{\Gamma \vdash E_1 : T_1, \Gamma \vdash E_2 : T_2}{\Gamma \vdash E_1, E_2 : T_1 \times T_2}$$

1. 类型声明如下所示，下列说法不正确的是：（ C ）

```
type    link = ↑cell;
        np = ↑cell;
        nqr = ↑cell;
var     next : link;

        last : link;
        p : np;
        q : nqr;
        r : nqr;
```

- A. 采用结构等价的观点看，next 和 p 类型相同。
- B. 采用名字等价的观点看，q 和 r 类型相同，p、q 类型不同。
- C. 采用结构等价的观点看，q 和 r 类型相同，p、q 类型不同。
- D. 采用名字等价的观点看，next 和 p 类型不同。

2. C 语言在进行类型检查时对所有类型都采用了结构等价的观点。 错

3. 如果类型转换由编译器自动完成，则称为显式类型转换。 错

4. 类型检查器通常根据类型系统的定义进行编写，类型系统中的定型环境 Γ 在类型检查器中通常用符号表来表示。 对

5. 类型检查器通常根据类型系统的定义进行编写，类型检查器中采用的所有类型在类型系统中都要有对应的类型表达式。 错

题解: type_error 在类型系统中没有对应的类型表达式。

1. 关于下面的程序，错误的说法是：（A）

```
void quickSort(int m, int n) {
    int i;

    if (n > m) {
```

```

    i = partition(m, n);
    quickSort(m, i-1);
    quickSort(i+1,n);

}

}

```

- A. 绿色 m,n 是形式参数。
- B. 蓝色 m, i-1 是实在参数。
- C. i 是局部于 quickSort 的变量。
- D. 红色 m,n 是形式参数。

2. 过程的活动记录中包含过程的代码。 错

3. 编译器为记录类型分配存储空间时，只要记录中域的数量、名称，及相应的类型不变，则无论记录域定义的顺序如何，为该记录分配的存储空间的大小不会变。 错

4. 调用序列全部处于调用过程中，返回序列全部处于被调用过程中。 错

题解：调用序列和返回序列分别有部分处于调用过程和被调用过程中。

5. 控制栈中当前存在的过程活动都是处在生存期内的活动。 对

1. 悬空引用是指引用某个被释放的存储单元。 对

2. 在无过程嵌套的静态作用域规则下，过程的活动要访问非局部变量需要用到访问链。 错

3. 在无过程嵌套的静态作用域规则下，过程可以作为参数来传递，也可以作为结果返回。 对

4. 使用值调用传递参数时，对形参的任何赋值会影响调用者的实参。 错

5. C 语言过程的活动记录中通常应该包含存放如下这些部分的区域：参数、返回地址、控制链、访问链、局部变量等。 错

1. 三地址码是语法树或 DAG 的一种线性表示，其中为语法树写出的三地址码通常比为 DAG 写出的三地址码更简洁（假设语法树和 DAG 都是为同样源程序写出的中间表示）。 错

2. 后缀表示通常用于处理表达式，其最大优点是便于用计算机处理表达式。 对
3. 赋值语句翻译成三地址码时，三地址码中的名字（变量名）实际上应该是该名字在符号表中的位置指针。 对
4. 声明语句翻译成三地址码时，主要是为声明的名字在符号表中建立相应条目并填写类型信息和相对地址信息等。 对
5. 对于允许过程嵌套的语言来说，所有声明的变量可以放在一个符号表中，用统一偏移计算相对地址。 错